AUTHENTICATION APPLICATION

MERN Authentication Application Development

Prepared By: Ritesh Chaudhary

Contents

1. Introduction **2**

2. Project Objectives **2**

3. Technology Stack **2**

4. System Overview and Architecture **3**

Frontend 3

Backend 3

Database 3

Application Architecture 3

5. Main System Modules and Implementation **3**

5.1 User Registration 3

5.2 User Login 3

5.3 Google Authentication 4

5.4 Forgot and Reset Password 4

5.5 Protected Routes 4

5.6 User Profile Management 4

6. Database Design **5**

7. Security Implementation **5**

8. Application Interface **5**

8.1 Home Page: 5

8.2 Sign-up and Sign-in Pages: 5

8.3 Forgot Password and Email Interface: 6

8.4 Profile Page: 6

9. Deployment **6**

10. Conclusion **6**

1. Introduction

The MERN Stack Authentication Application is a full-stack web application developed to implement user authentication and account management. It allows users to create accounts, sign in using their credentials or Google authentication, manage their profiles, and reset forgotten passwords through email.

The application uses React for the frontend, Node.js and Express.js for backend development, and MongoDB for storing user information. It also uses Redux Toolkit for frontend state management and JSON Web Tokens (JWT) for authentication.

The project demonstrates the practical implementation of frontend and backend integration, database operations, authentication, password security, and application deployment.

2. Project Objectives

The main objectives of this project are:

- To develop a full-stack web application using the MERN Stack.
- To implement user registration and login functionality.
- To provide authentication using email and password, as well as Google sign-in.
- To protect user accounts and restricted application pages.
- To implement password hashing and JWT-based authentication.
- To develop a password recovery process using email.
- To allow users to manage their profile information.
- To deploy the application for public access.

3. Technology Stack

The following technologies and tools were used to develop the application.

Category	**Technologies**
Frontend	React, JavaScript, Tailwind CSS, Vite
Routing and State Management	React Router, Redux Toolkit
Backend	Node.js, Express.js
Database	MongoDB, Mongoose
Authentication	JSON Web Token (JWT), bcryptjs
Email Service	Nodemailer, Gmail SMTP
Deployment and Version Control	Render, Git, GitHub

4. System Overview and Architecture

The application follows a client-server architecture. The frontend handles user interaction, while the backend processes requests, manages authentication, and communicates with the MongoDB database.

The main components are:

Frontend: Developed using React and Tailwind CSS. It provides the user interface for registration, login, password recovery, dashboard access, and profile management. React Router manages navigation, while Redux Toolkit manages application and user authentication states.

Backend: Developed using Node.js and Express.js. It provides API endpoints for authentication and user-related operations. The backend validates requests, processes user information, and communicates with MongoDB.

Database: MongoDB stores user account information. Mongoose is used to define the user model and perform database operations.

Application Architecture

- User Interface (React)
- API Requests (Express.js)
- Authentication and User Controllers
- MongoDB Database (Mongoose)

External services are also integrated for Google authentication and sending password-reset emails.

5. Main System Modules and Implementation

5.1 User Registration

The registration module allows new users to create an account by providing a username, email address, and password.

The backend checks whether the username or email address already exists. The password is hashed using bcryptjs before the new user information is saved in MongoDB.

5.2 User Login

The login module authenticates registered users using their email address and password.

The backend retrieves the user account from MongoDB and compares the submitted password with the stored hashed password using bcryptjs.

After successful authentication, a JWT is generated and stored in an HTTP-only cookie. The frontend updates the authentication state and provides access to the protected application pages.

5.3 Google Authentication

The application provides Google sign-in as an alternative authentication method.

When a user signs in through Google, the backend checks whether an account with the supplied email address already exists. If an account exists, it creates an authentication token for that user. Otherwise, it creates a new user account and stores the associated information in MongoDB.

5.4 Forgot and Reset Password

The password recovery module allows users to reset their passwords if they cannot remember their existing password.

The process works as follows:

1. The user submits their registered email address.
2. The backend checks whether the account exists in MongoDB.
3. A JWT reset token is generated with a validity period of three minutes.
4. The token is stored in the user's database record.
5. Nodemailer sends an email containing a password-reset link.
6. The user opens the link and the backend validates the user and token.
7. After successful validation, the user can submit a new password.
8. The new password is hashed using bcryptjs and updated in MongoDB.

5.5 Protected Routes

The application uses a private route component with React Router and Redux Toolkit.

The private route checks the current authentication state. Authenticated users can access restricted pages such as the dashboard, while unauthenticated users are redirected to the sign-in page.

5.6 User Profile Management

The profile module allows authenticated users to view and manage their account information.

The profile interface provides options to update profile information, change the password, delete the account, and log out. It also displays the user's profile picture and account details.

6. Database Design

The application uses MongoDB to store user account information. The user collection is managed through a Mongoose model.

The main user fields are:

Fields	Description
username	Stores the user's username.
email	Stores the user's email address.
password	Stores the hashed password.
profilePicture	Stores the user's profile picture information.
verifyToken	Stores the password-reset verification token.

The database supports user registration, authentication, profile management, and password recovery operations.

7. Security Implementation

The application includes several security-related mechanisms:

- **Password Hashing:** bcryptjs is used to hash passwords before storing them in MongoDB.
- **JWT Authentication:** JSON Web Tokens are used to identify authenticated users.
- **HTTP-Only Cookies:** Authentication tokens are stored in HTTP-only cookies.
- **Password Reset Token:** A time-limited JWT is used to validate password-reset requests.
- **Environment Variables:** Sensitive configuration values, such as database credentials, JWT secrets, and email credentials, are maintained in environment variables.

8. Application Interface

The application provides a simple and responsive interface for user authentication and account management.

The main interfaces include:

8.1 Home Page:

Provides the application's main landing page and navigation options.

8.2 Sign-up and Sign-in Pages:

Allow users to register and authenticate using their account credentials.

8.3 Forgot Password and Email Interface:

Allows users to request a password-reset link and receive it through email.

8.4 Profile Page:

Displays user information and provides account-management options.

Application screenshots are included below to demonstrate the implemented interfaces.

9. Deployment

The application is deployed on Render and its source code is maintained in GitHub.

The deployment process involves pushing code changes to the connected GitHub repository. Render then builds and deploys the latest code using the configured build and start commands.

Production environment variables are configured through the Render dashboard rather than being stored in the public source-code repository.

Live Application: https://mern-auth-app-rm8f.onrender.com/
Source Code: https://github.com/riteshchaudhary375/mern-auth-app

10. Conclusion

The MERN Stack Authentication Application demonstrates the development of a full-stack web application using modern JavaScript technologies.

Through this project, I implemented user registration, login, Google authentication, password hashing, JWT-based authentication, password recovery through email, protected routes, and profile management.

The project provided practical experience in developing REST API functionality, connecting a React frontend with an Express backend, performing database operations using MongoDB and Mongoose, managing application state with Redux Toolkit, and deploying a full-stack application using Render.

Overall, this project helped strengthen my understanding of full-stack development, backend integration, database management, and authentication processes.